Agricultural Equipment and Asset Management System

A system developed to manage agricultural machinery, equipment, and other municipal agricultural assets. It supports equipment reservations, borrowing and return transactions, maintenance scheduling, equipment availability monitoring, and asset utilization tracking to improve operational efficiency.

Roles: Admin and Employee | **Data platform:** centralized PostgreSQL

Theme: green and teal | **Layout:** one diagram per page, with notes and notation

01 System Architecture

02 Class Diagram

03 Component Diagram

04 Deployment Diagram

05 Object Diagram

06 Package Diagram

07 Composite Structure Diagram

08 Profile Diagram

09 Data Flow Diagram

10 Use Case Diagram

11 Activity Diagram

12 Sequence Diagram

13 State Machine Diagram

14 Communication Diagram

15 Interaction Overview Diagram

16 Timing Diagram

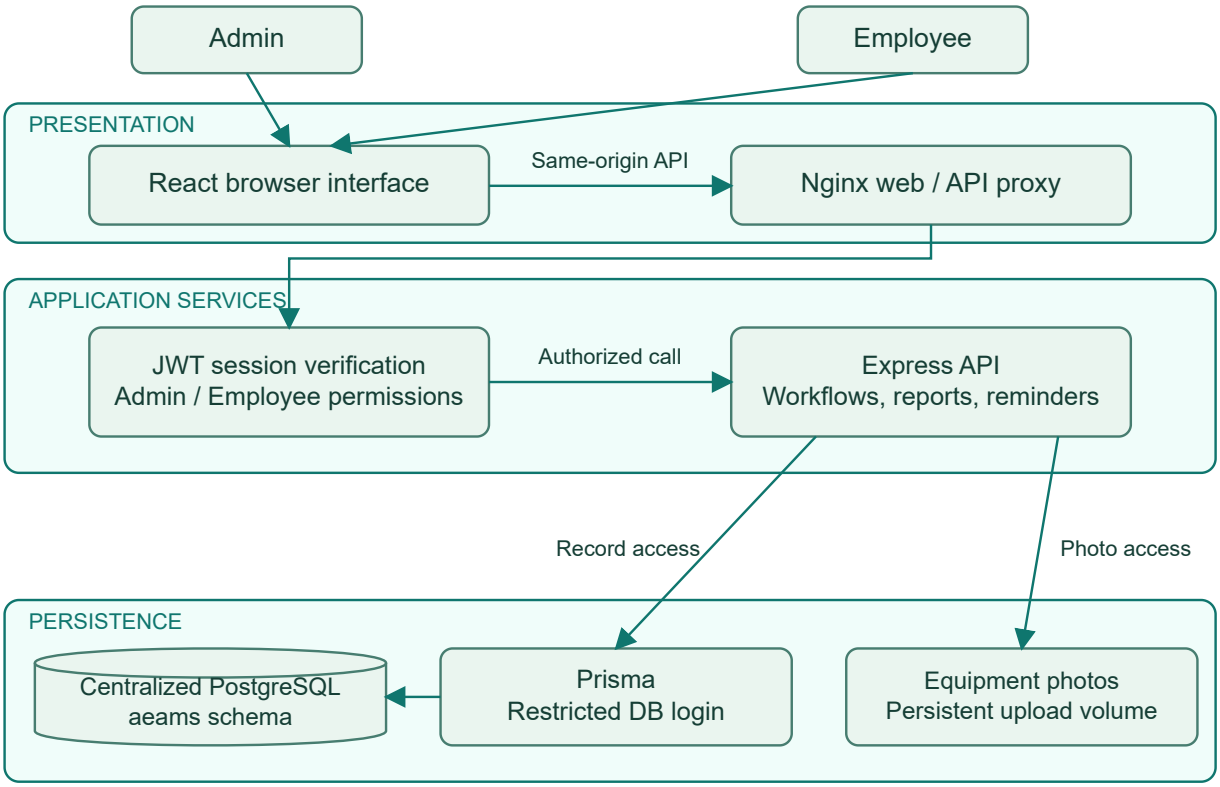
17 Entity Relationship Diagram

18 SysML Requirement Diagram

Based on the implemented AEAMS model, permissions, and workflows. Examples are fictional. Modeling-only conventions, omitted supporting entities, and the optional HTTPS deployment path are identified where applicable.

01 · System Architecture

The implemented React → Nginx → Express → Prisma → PostgreSQL stack. Farmers are borrower references, not portal users.

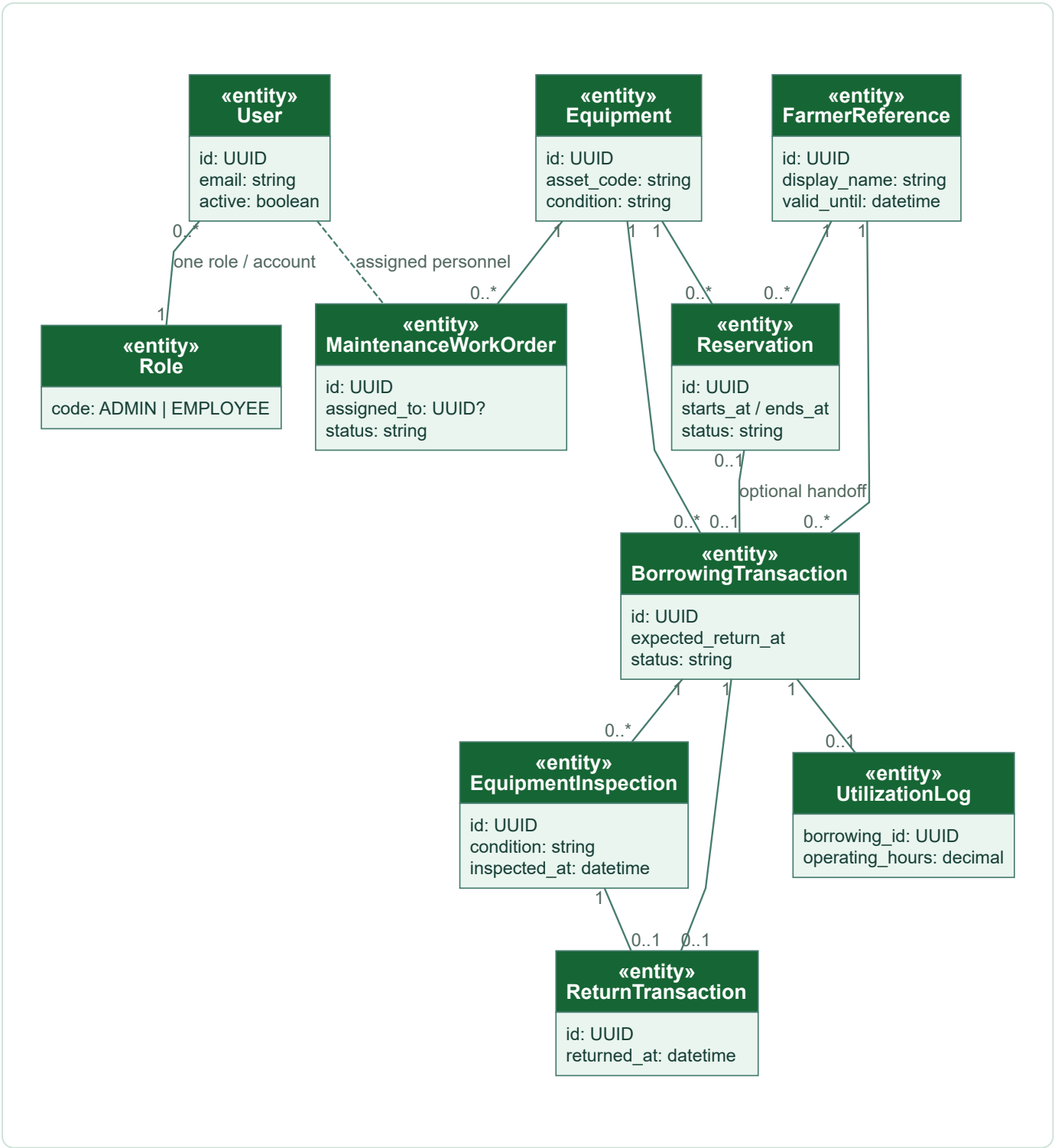


HOW TO READ THIS DIAGRAM

- Requests pass from React through Nginx to the Express API; JWT cookies identify the signed-in staff member.
- The API checks permissions before equipment, booking, maintenance, and reporting operations.
- PostgreSQL stores records and audit evidence; a separate volume holds authenticated equipment photos.

02 · Class Diagram

Conceptual domain model aligned with the implemented records. User roles are Admin or Employee; workflow operations are responsibilities, not literal ORM methods.

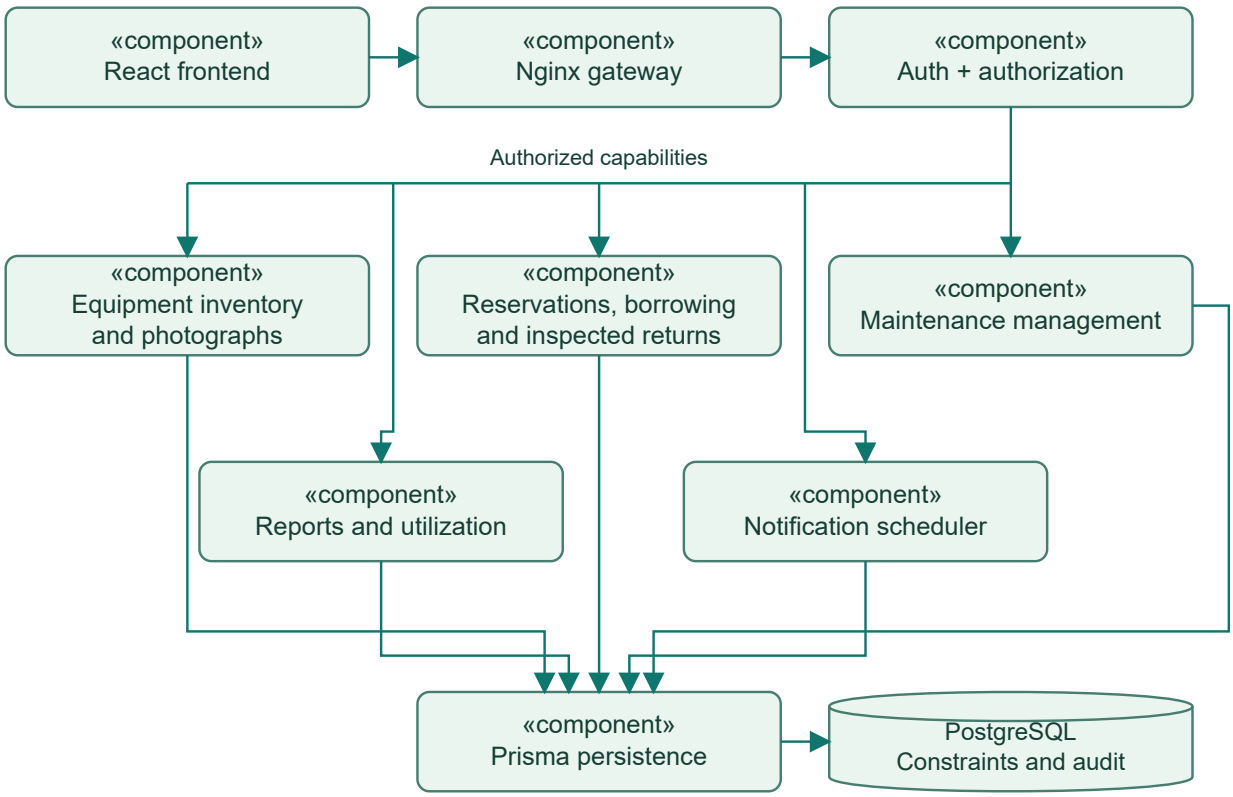


HOW TO READ THIS DIAGRAM

- Each staff account has one role: Admin or Employee. One role can be assigned to many accounts.
- Equipment and verified farmer references link to reservations and borrowing transactions.
- A borrowing may have one return and one utilization log; inspections supply return-condition evidence.

03 · Component Diagram

Logical components and their dependencies. Database constraints enforce conflicts and evidence requirements in addition to API validation.

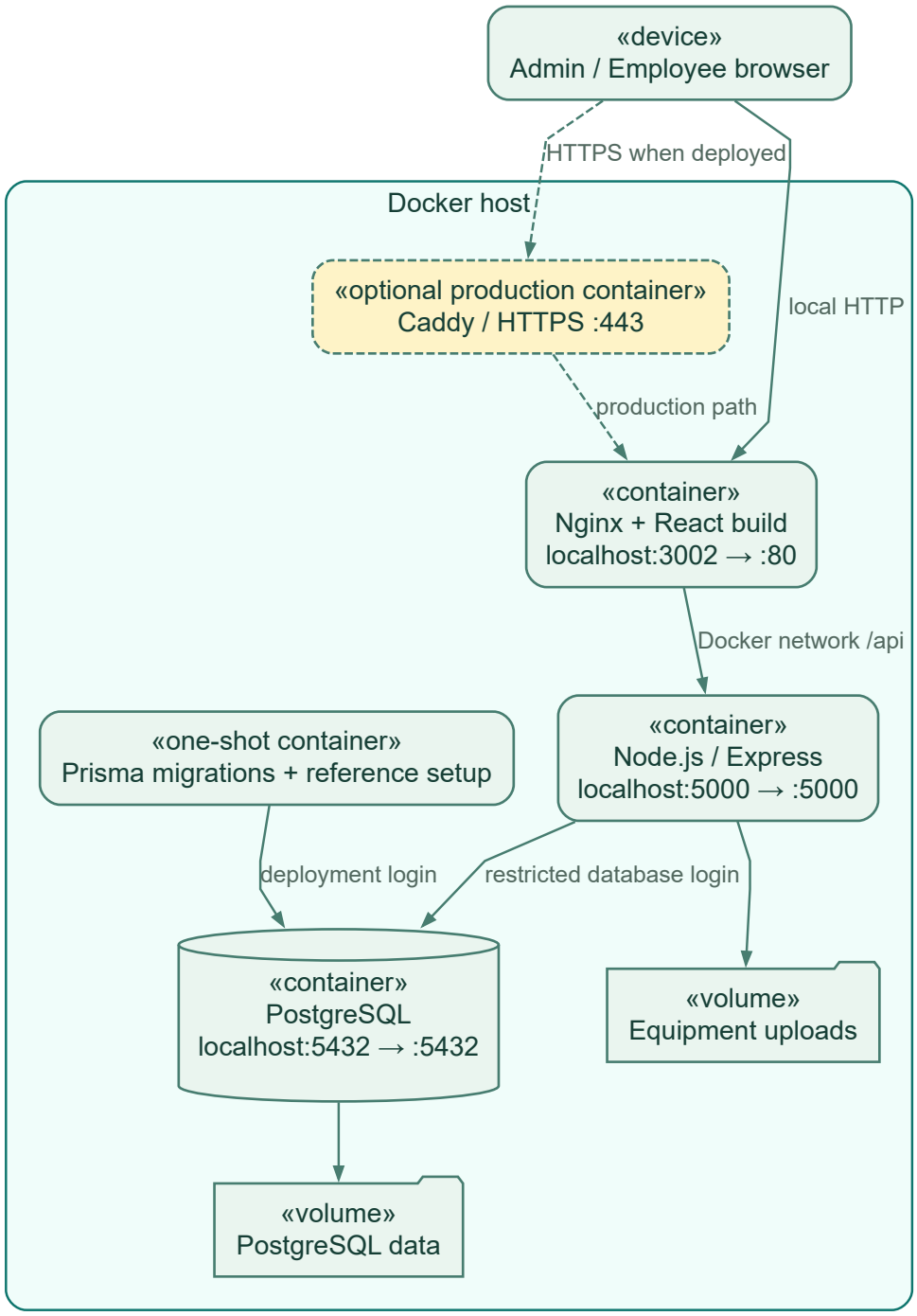


HOW TO READ THIS DIAGRAM

- Frontend and proxy components call authenticated API capabilities through a same-origin endpoint.
- Inventory, transactions, maintenance, reports, and reminders depend on the persistence component.
- Database constraints supplement API validation; permissions are enforced on the backend.

04 · Deployment Diagram

Current local Docker deployment, with the configured HTTPS gateway shown as a production option. Database/photo volumes persist across container replacement.

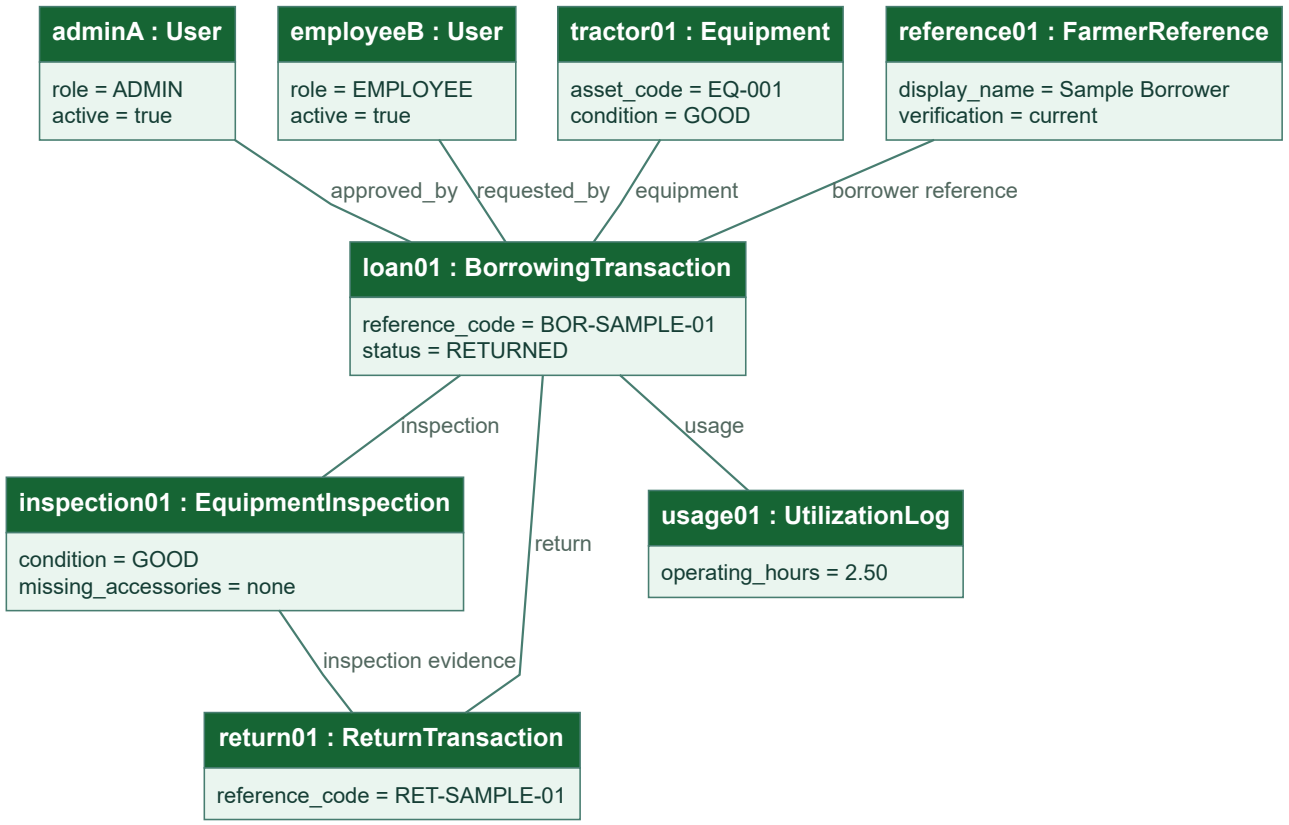


HOW TO READ THIS DIAGRAM

- The local browser reaches Nginx on port 3002; Nginx proxies API calls to the backend container.
- The API uses a restricted database login. The migration job uses a separate deployment credential.
- Database and photo volumes persist. The dashed Caddy path is a configured production HTTPS option.

05 · Object Diagram

Fictional snapshot after a good inspected return. Object values illustrate valid relationships and are not personal or live municipal records.

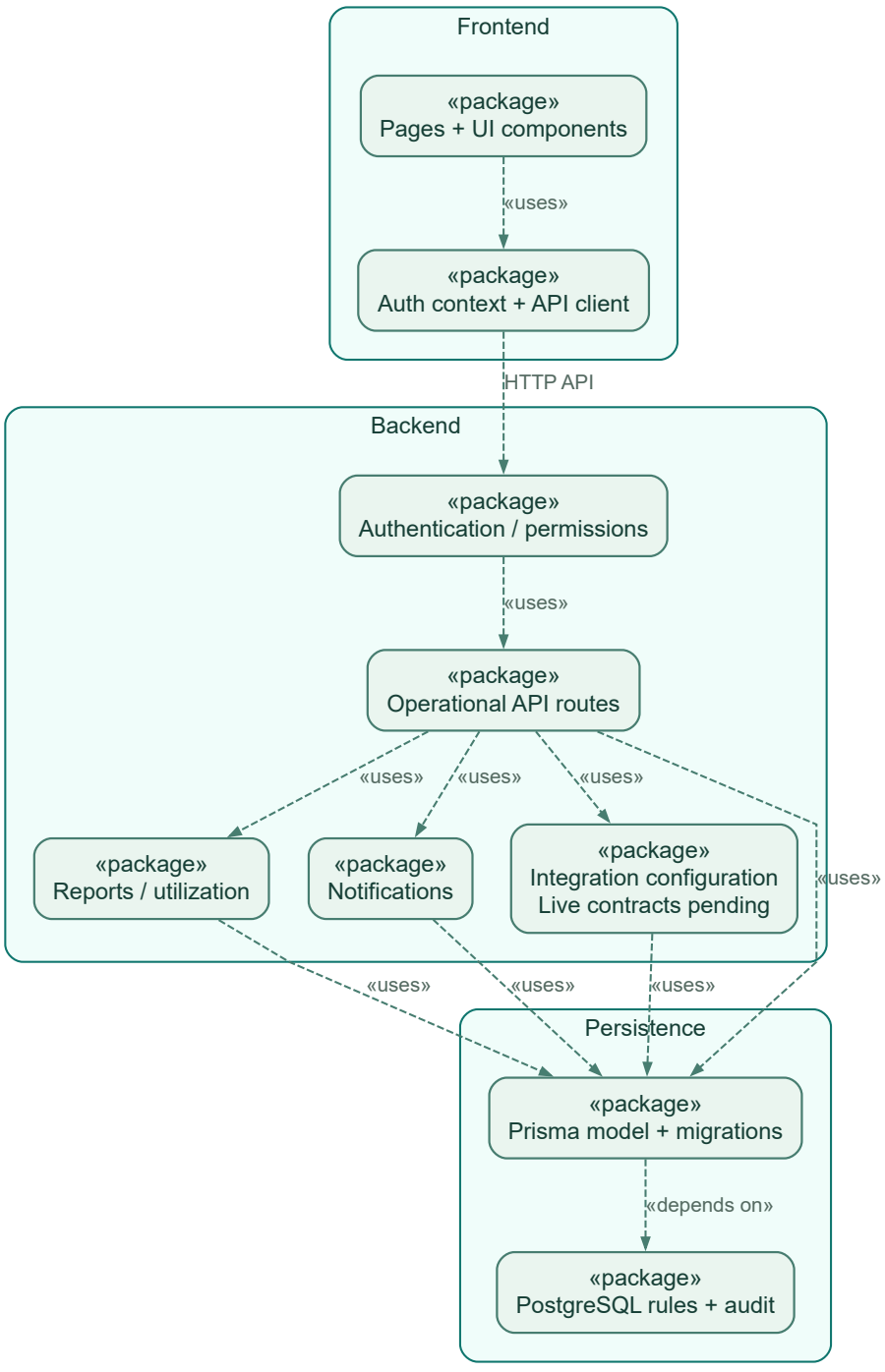


HOW TO READ THIS DIAGRAM

- This fictional snapshot shows a borrowing that has been returned after a good-condition inspection.
- Admin A supplies approval; Employee B requests the loan. Both refer to the same transaction.
- The return references its inspection, and the utilization log records 2.50 operating hours.

06 · Package Diagram

Logical organization of the implemented frontend, API and persistence responsibilities; package names express design groupings.

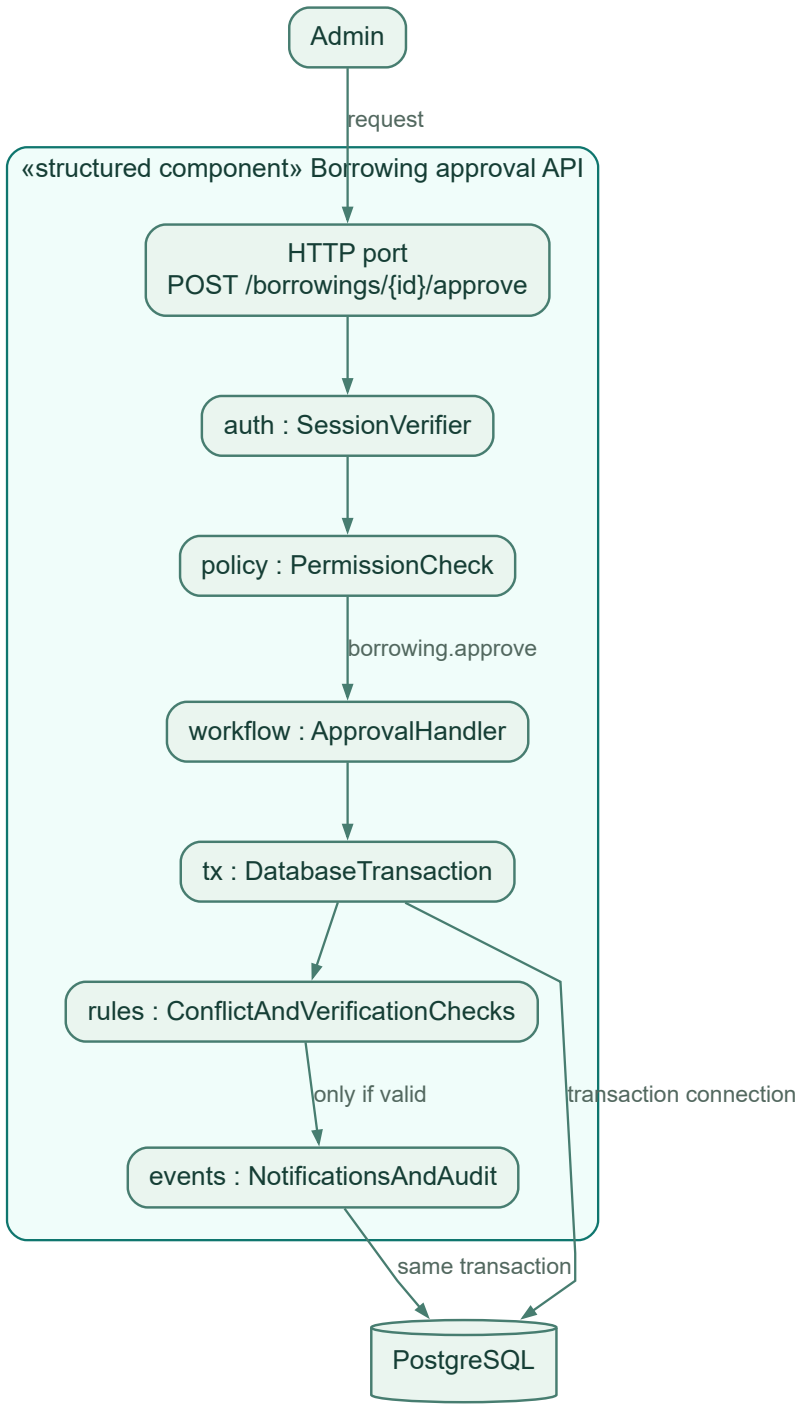


HOW TO READ THIS DIAGRAM

- Presentation packages contain pages, reusable UI components, authentication context, and API access.
- Backend responsibilities are grouped into security, routes, reports, notifications, and integrations.
- Dashed dependencies lead to Prisma and PostgreSQL. External module delivery remains unconfigured.

07 · Composite Structure Diagram

Internal parts of a borrowing-approval request. A booking conflict rolls back the transaction; notifications and audit evidence commit with the change.

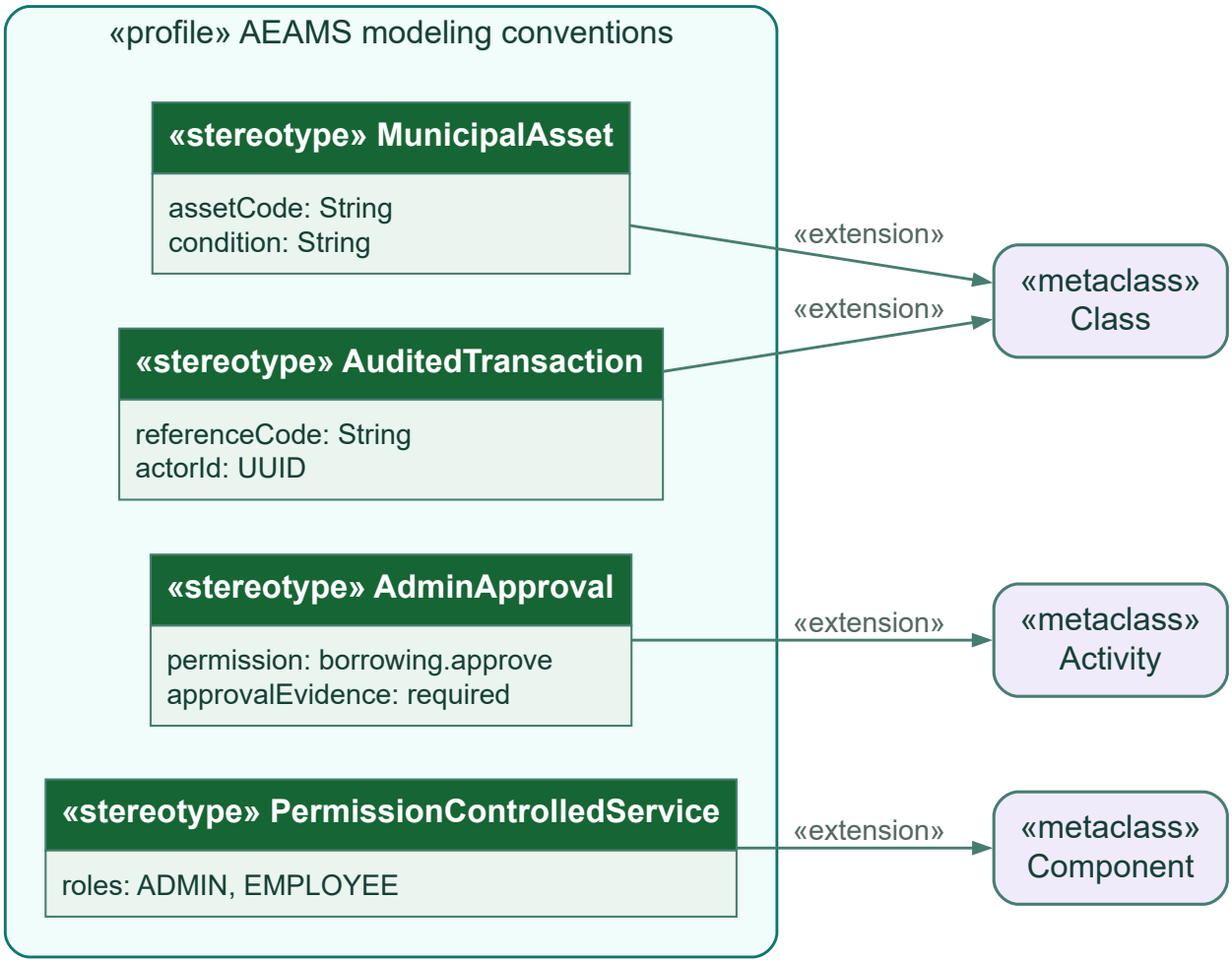


HOW TO READ THIS DIAGRAM

- An approval request enters through the HTTP port and passes session and permission checks.
- The handler opens an actor-scoped transaction and validates borrower and equipment booking rules.
- Approval, notification, and audit evidence commit together; a failed constraint rolls back the change.

08 · Profile Diagram

Proposed UML modeling conventions for this documentation. These stereotypes are design annotations, not an installed runtime framework or additional user roles.

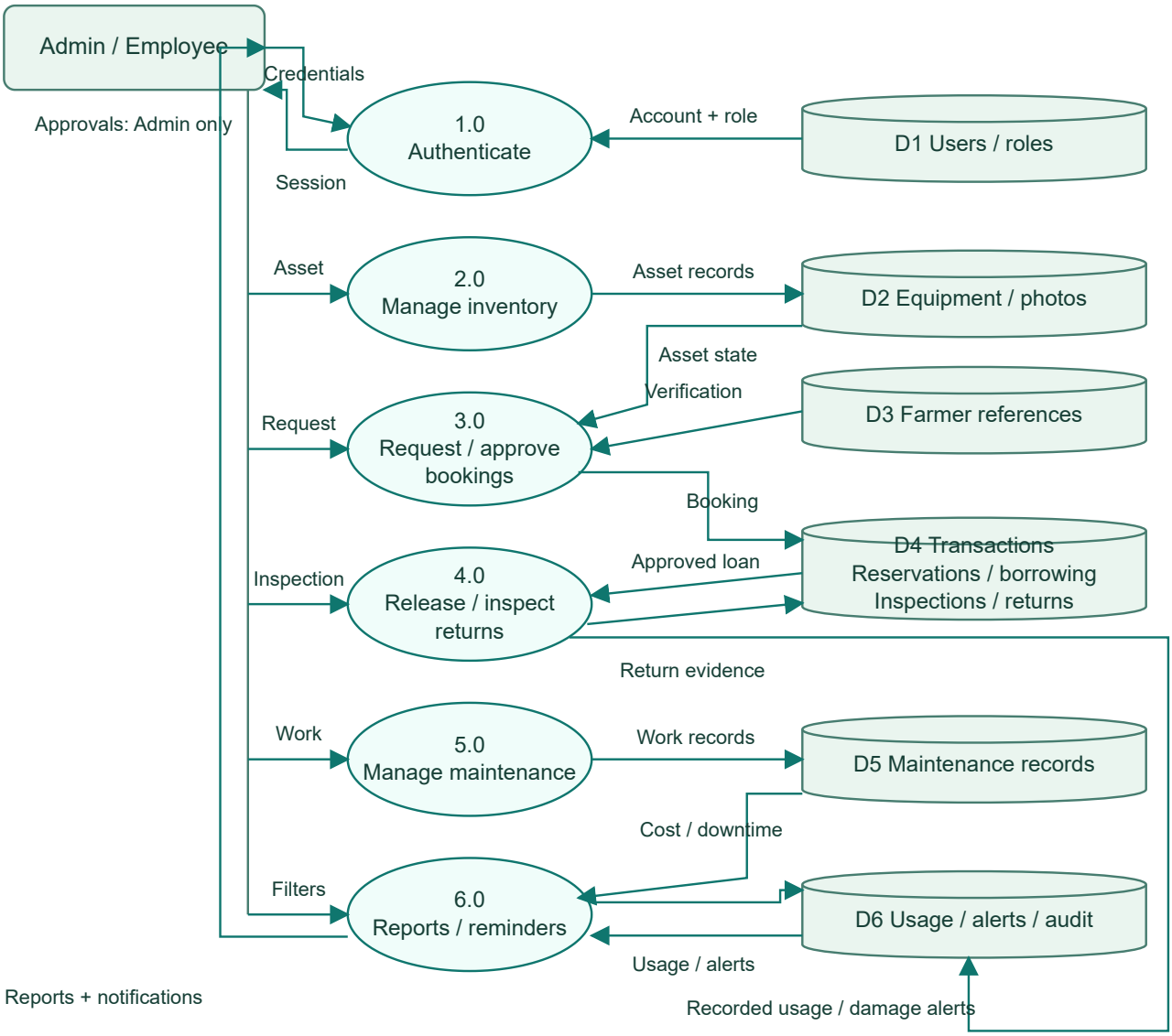


HOW TO READ THIS DIAGRAM

- The profile proposes documentation stereotypes for assets, transactions, reviews, and services.
- Each extension arrow connects a proposed stereotype to its UML metaclass.
- These modeling conventions are annotations, not runtime modules or additional portal roles.

09 · Data Flow Diagram

Level 1 DFD: rectangles are external staff, circles are processes, cylinders are logical stores. Process labels map to actual AEAMS responsibilities.

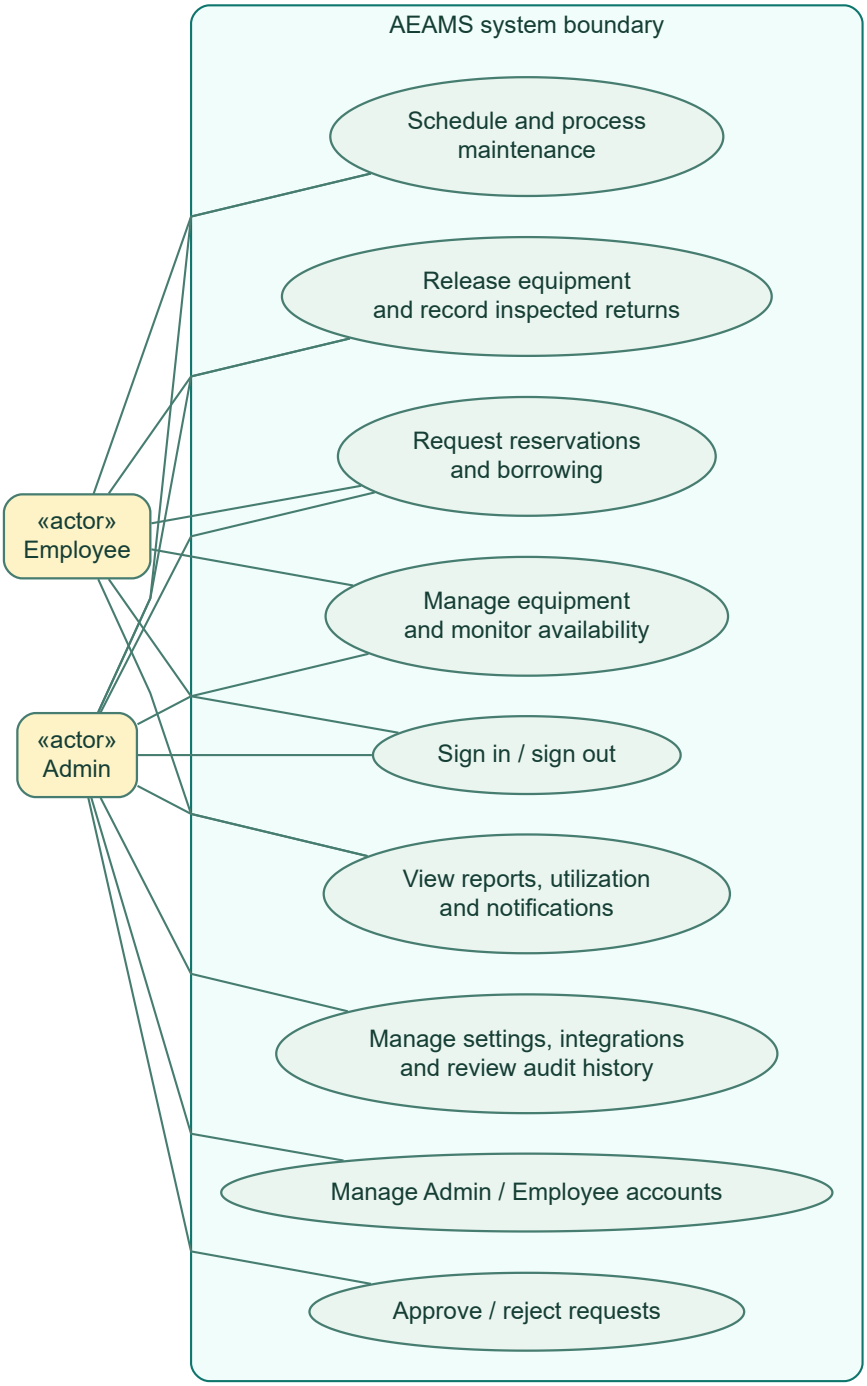


HOW TO READ THIS DIAGRAM

- Rectangles represent external staff, circles are processes, and cylinders are logical data stores.
- Request approval uses verified references and equipment availability before storing a booking.
- Inspected returns feed transaction history, asset condition, usage, maintenance alerts, and reports.

10 · Use Case Diagram

Exactly two portal actors. Admin manages approvals and administration; both roles carry out permitted equipment operations. Farmers remain external borrower references.

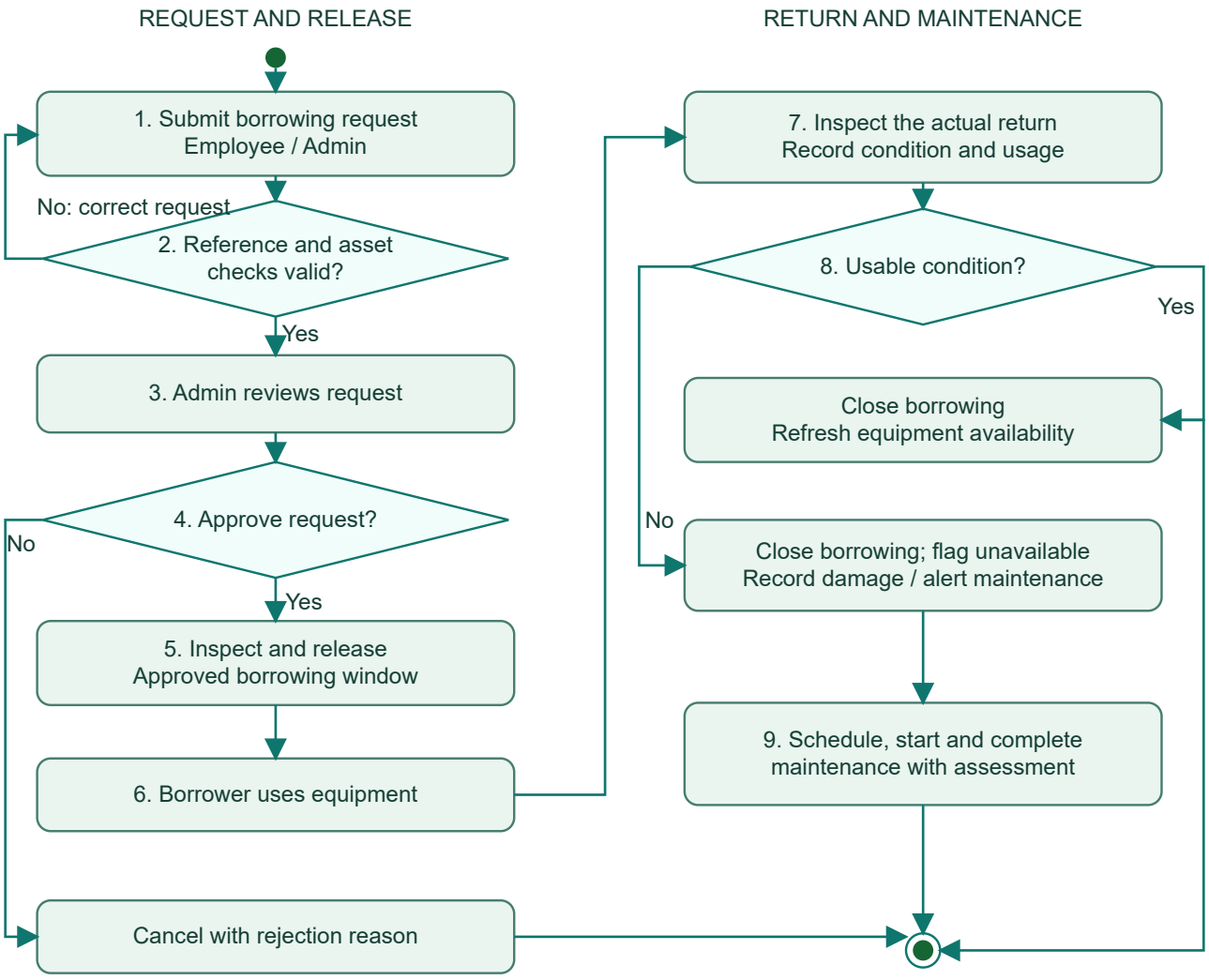


HOW TO READ THIS DIAGRAM

- Admin and Employee share sign-in, inventory, requests, release/return, maintenance, and reporting.
- Only Admin approves or rejects requests and manages accounts, integrations, settings, and audit access.
- Associations show permitted participation; farmers are external borrower references, not portal users.

11 · Activity Diagram

End-to-end operational workflow. The Admin decides approval; permitted staff release and inspect the return. Damage flags the asset unavailable and alerts maintenance.

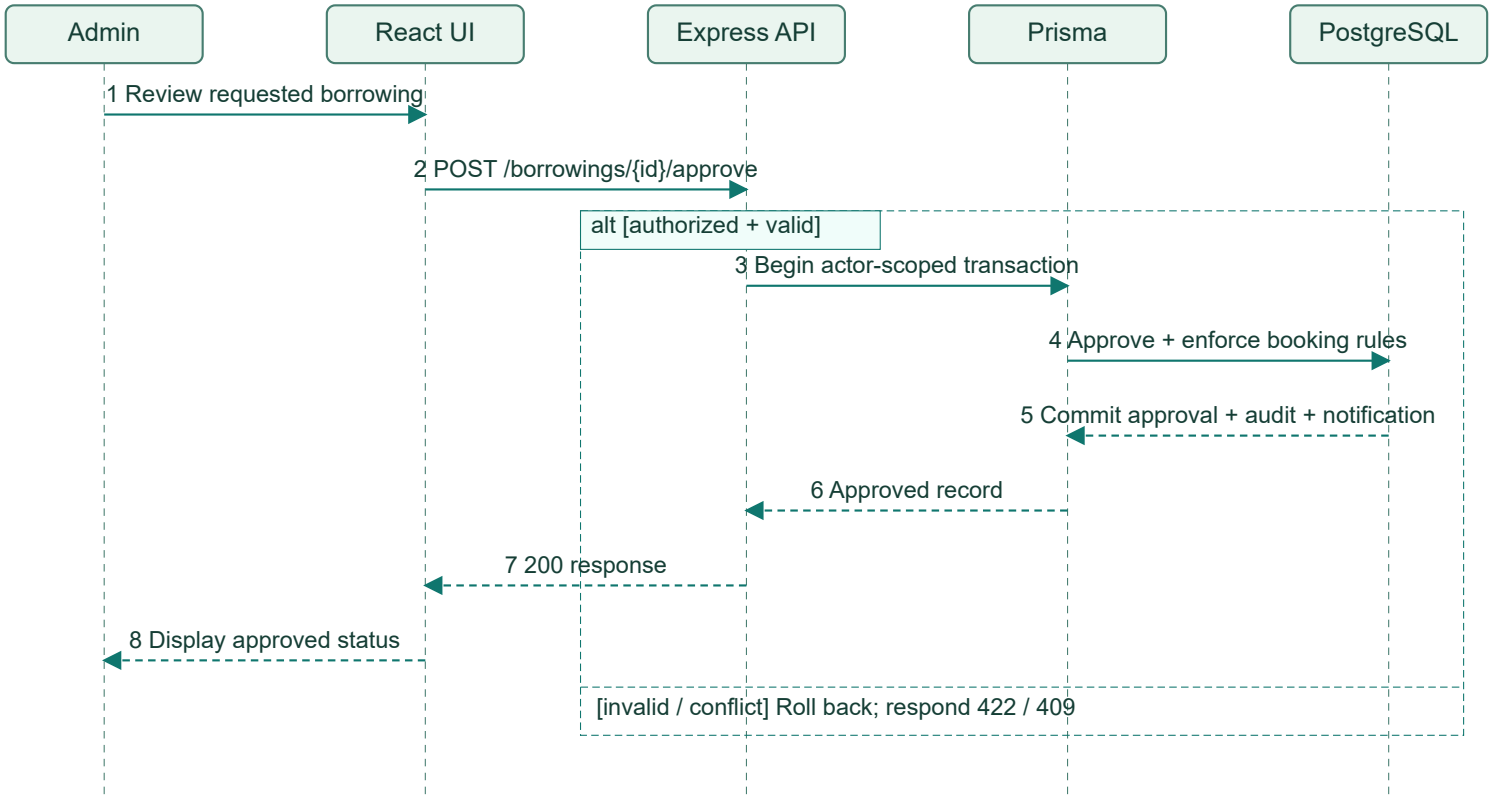


HOW TO READ THIS DIAGRAM

- A request is corrected if reference or equipment checks fail; Admin then approves or rejects it.
- Permitted staff inspect and release approved equipment, then inspect it again when returned.
- A usable return refreshes availability. Damage flags the asset unavailable and starts maintenance review.

12 · Sequence Diagram

Borrowing approval sequence. Authentication and permission checks precede the database transaction; constraint failures roll back instead of committing partial approvals.

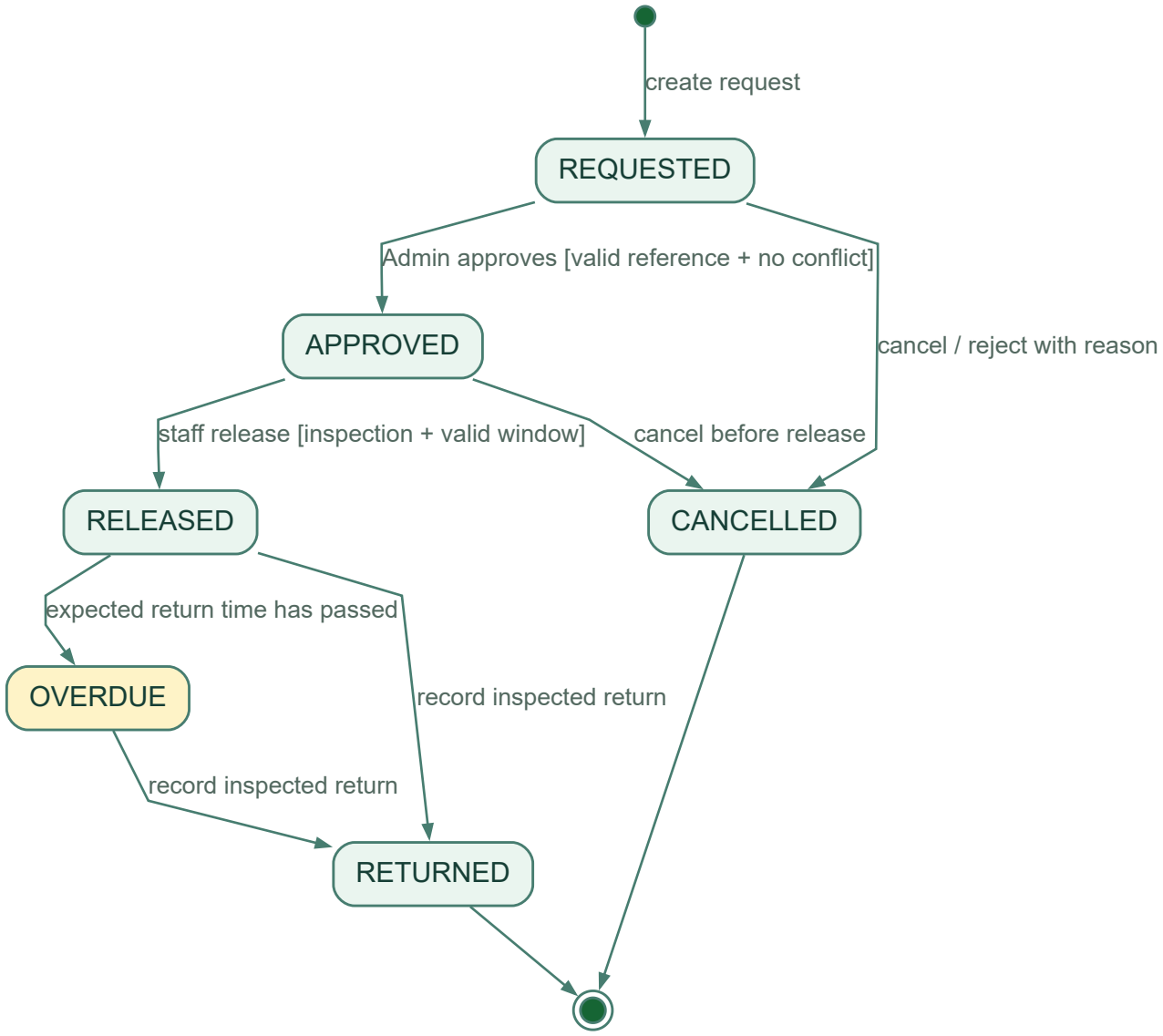


HOW TO READ THIS DIAGRAM

- Lifelines run downward in time; solid arrows are requests and dashed arrows are responses.
- Approval uses an actor-scoped Prisma transaction and PostgreSQL booking/verification constraints.
- The alternative fragment distinguishes a committed approval from rollback with a 422 or 409 response.

13 · State Machine Diagram

Canonical BorrowingTransaction status values from the database. UI labels may show Requested as Pending and Released as Borrowed. Rejection is stored as CANCELLED with a reason.

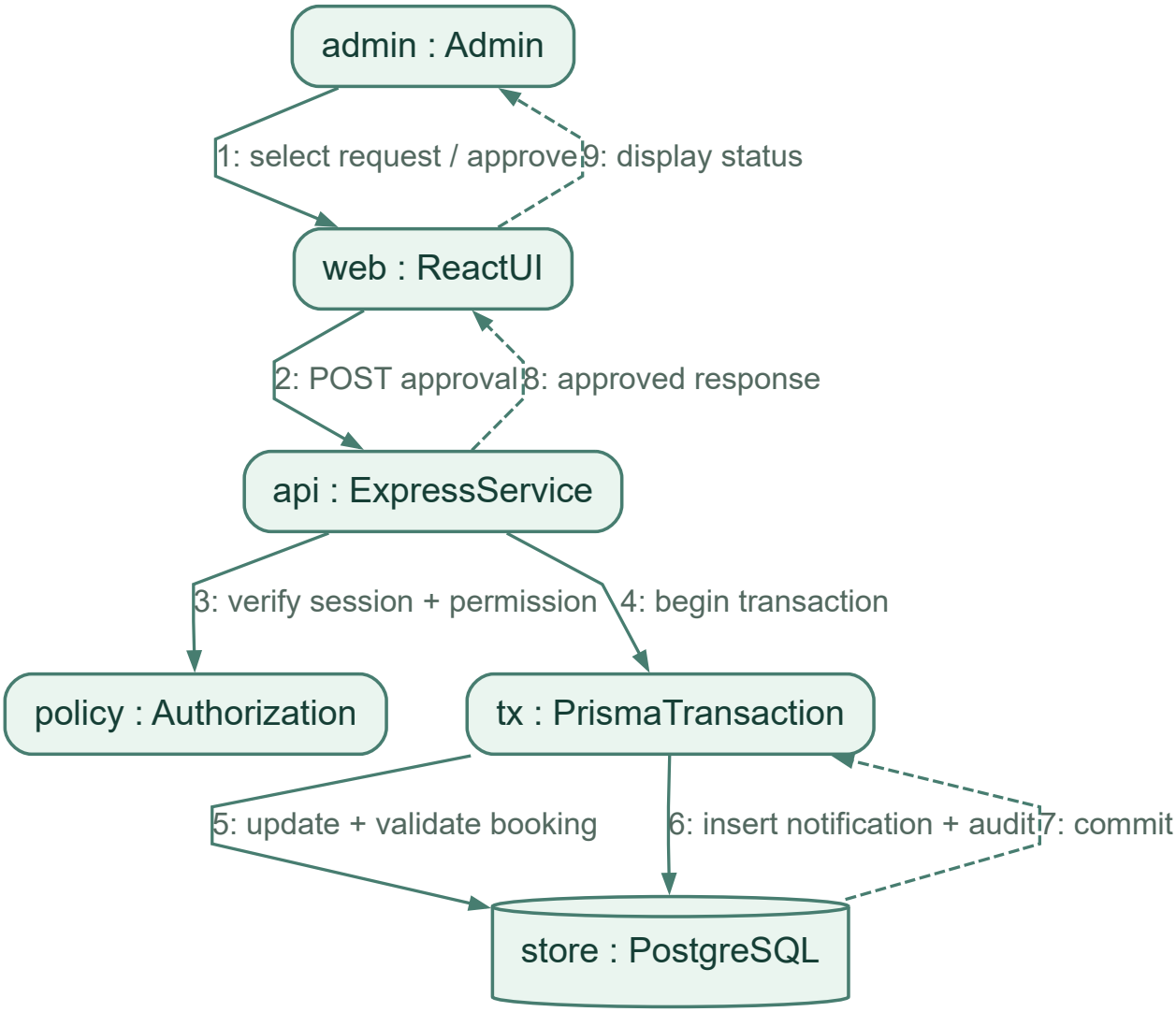


HOW TO READ THIS DIAGRAM

- REQUESTED moves to APPROVED only after Admin approval; rejected requests become CANCELLED.
- Release requires inspection evidence and a valid window. An unreturned released loan may become OVERDUE.
- An inspected return closes RELEASED or OVERDUE as RETURNED. Cancellation is allowed before release.

14 · Communication Diagram

Numbered messages for a borrowing approval. The notification and audit records are written inside the same transaction as the approved borrowing.

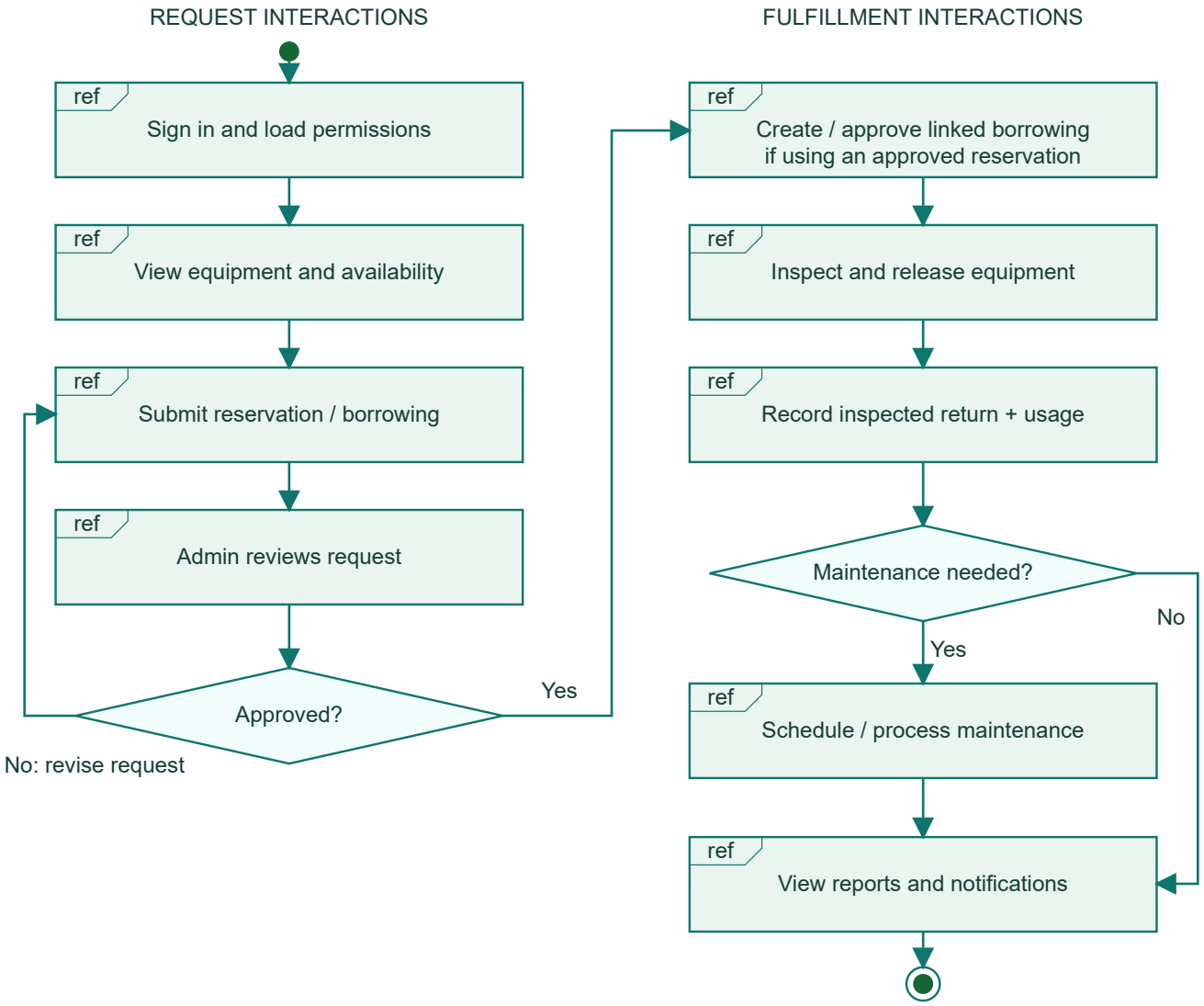


HOW TO READ THIS DIAGRAM

- Messages are numbered to show the order of collaboration among the participating objects.
- The API verifies authorization before opening the database transaction and applying booking rules.
- Approval, notification, and audit writes commit together; the UI then displays the approved result.

15 · Interaction Overview Diagram

High-level orchestration of the system interactions. Each ref node represents a separate user/API interaction, not an additional deployed service.

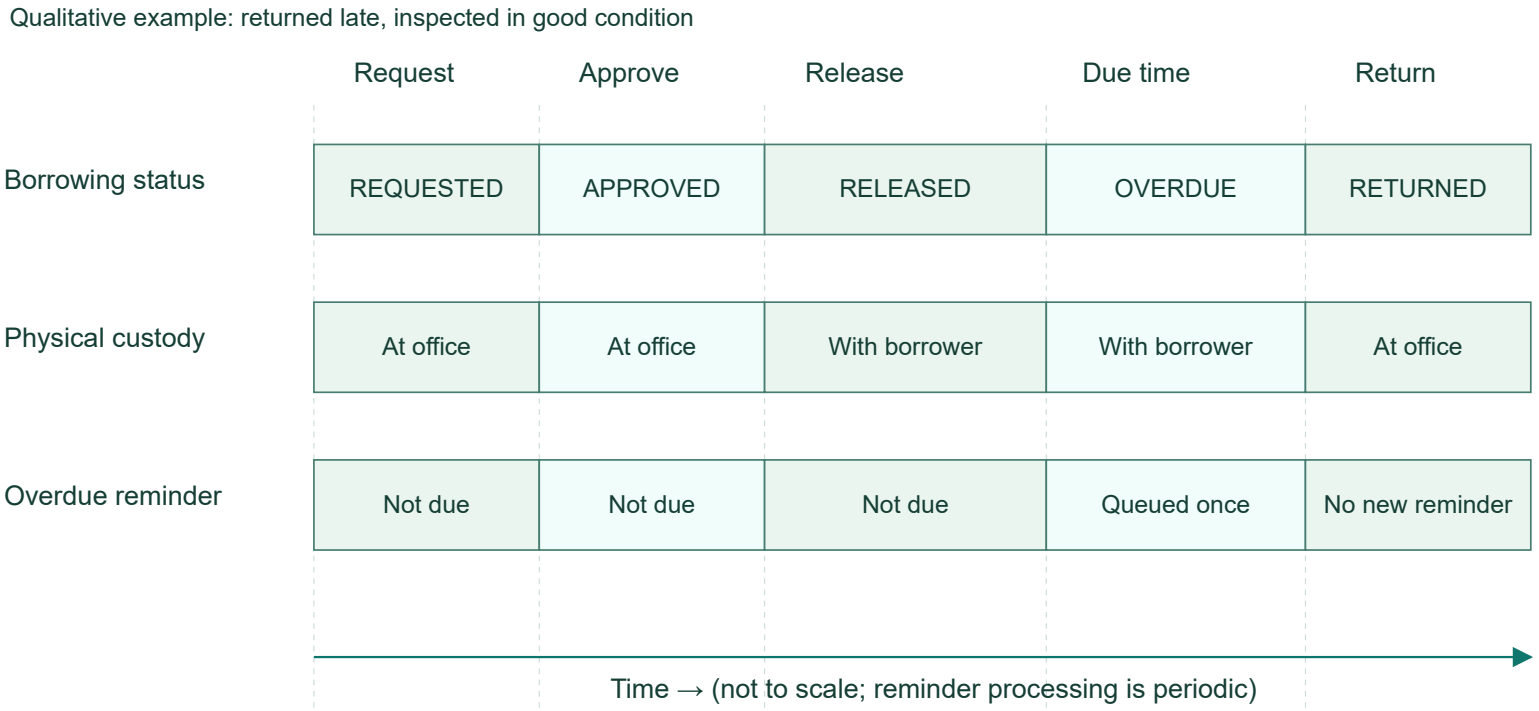


HOW TO READ THIS DIAGRAM

- Each ref frame summarizes an interaction; diamonds select approval and maintenance paths.
- When a reservation is used, a linked borrowing must be created and approved before equipment release.
- Rejection returns to request revision; inspected returns lead to maintenance when required, then reports.

16 · Timing Diagram

Qualitative timing, not a measured response-time or SLA claim. OVERDUE is set by the reminder process after the expected return time; reminders are deduplicated.



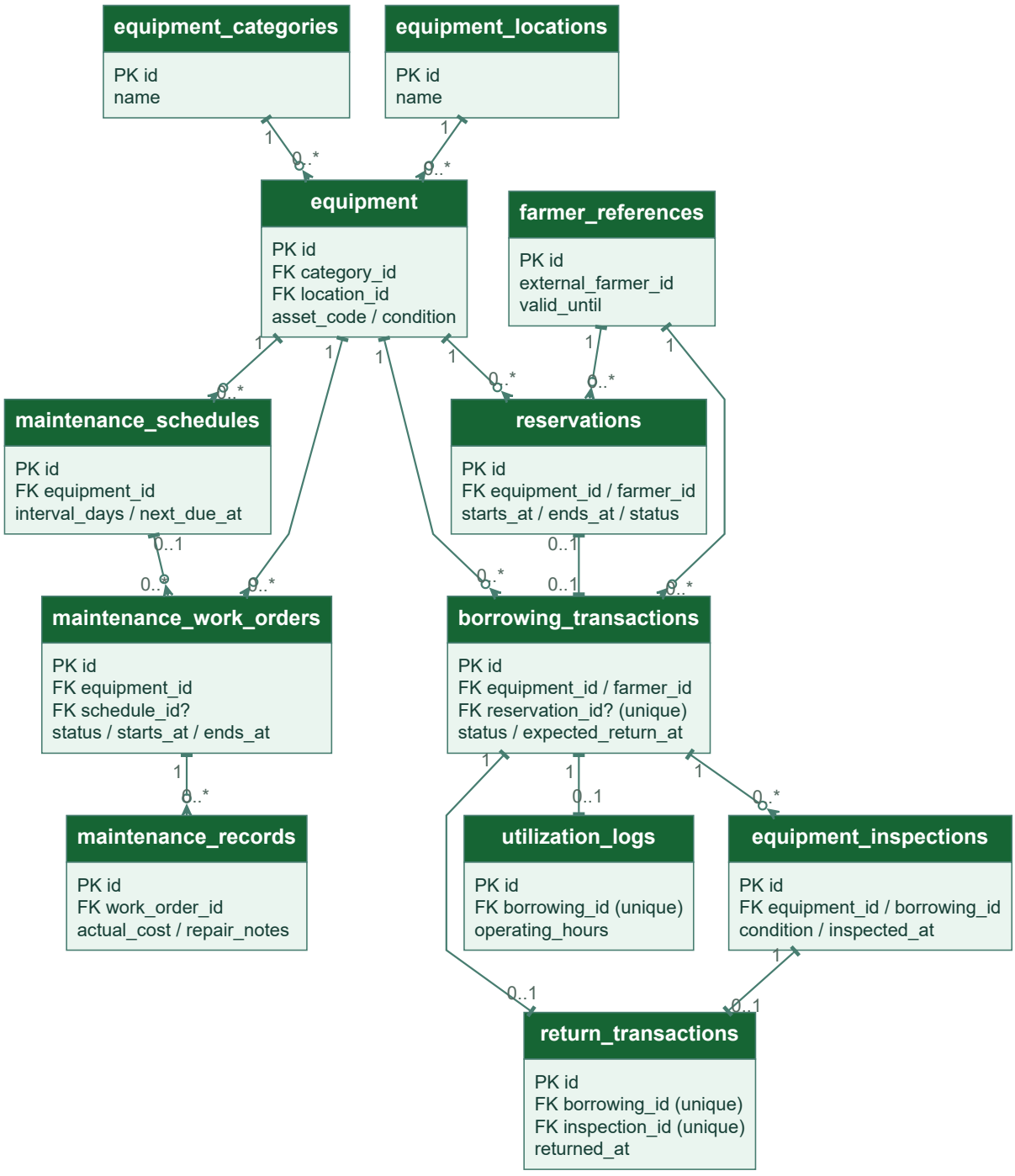
Active borrowing blocks new availability until an inspected return.
A future booking is different from physical custody.

HOW TO READ THIS DIAGRAM

- The phases illustrate request, approval, release, due time, and a late inspected return.
- Borrowing status changes while custody remains with the borrower until the physical return.
- Overdue reminders are periodic and deduplicated. This is qualitative timing, not a performance SLA.

17 · Entity Relationship Diagram

Core physical entities only; supporting roles/permissions, audit, bookings, settings, damage, and integration tables are omitted for readability. PK = primary key; FK = foreign key; ? = nullable.

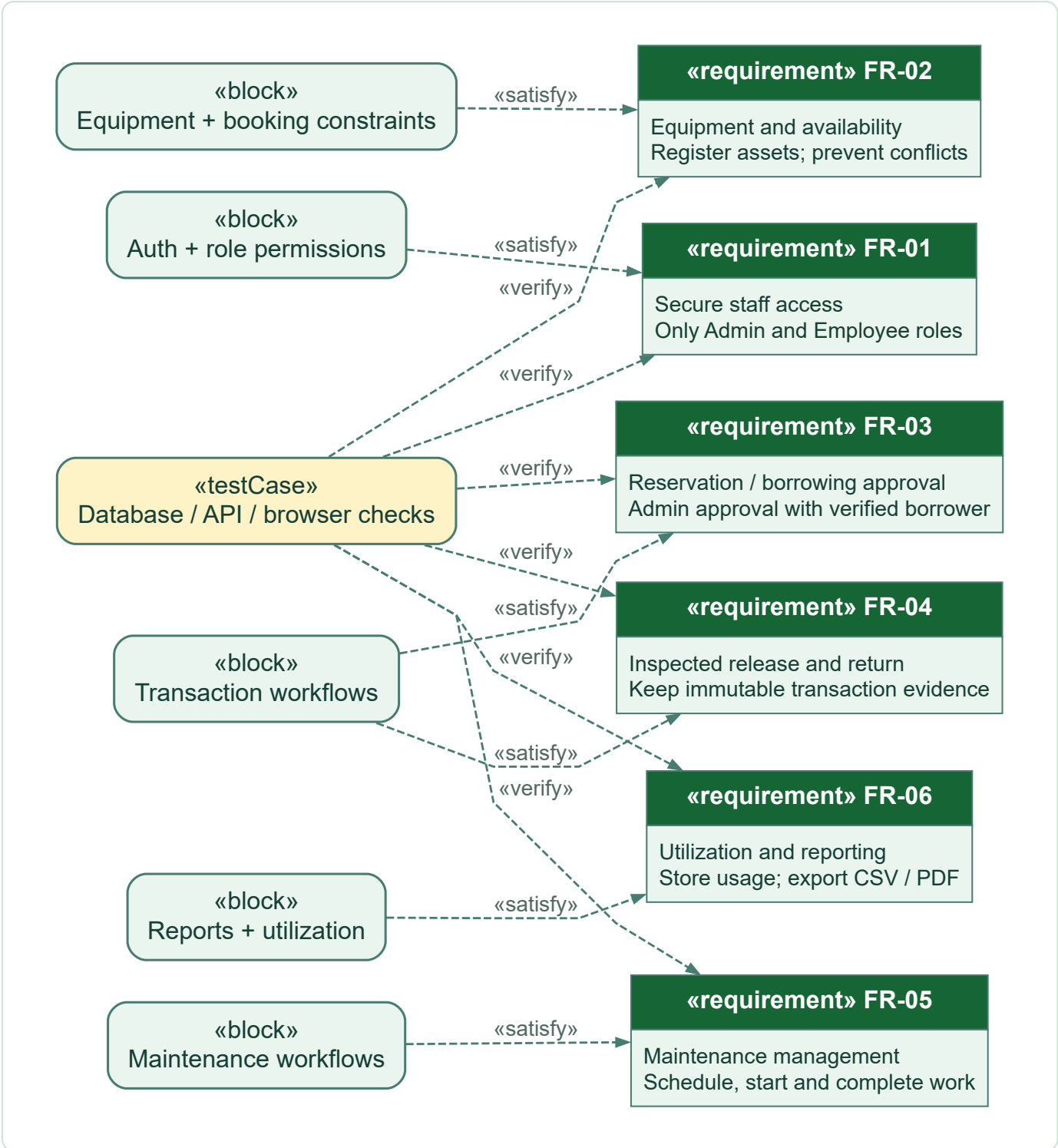


HOW TO READ THIS DIAGRAM

- PK identifies a primary key; FK identifies a foreign key. A question mark denotes a nullable field.
- A reservation may link to one borrowing. Each borrowing has at most one return and one usage log.
- This core ERD omits supporting audit, authorization, booking, damage, settings, and integration tables.

18 · SysML Requirement Diagram

Requirements are traced to implemented responsibilities and verification activities. This is a focused system-level traceability view, not a claim of external certification.



HOW TO READ THIS DIAGRAM

- FR-01 through FR-06 cover staff access, inventory, approvals, inspections, maintenance, and reporting.
- Dashed satisfy links map implementation responsibilities to the requirements they address.
- Dashed verify links map database, API, and browser checks to requirements; this is not certification.